INTERFACE AUDIT · NWU · 2026

DACMS / AACMS

Design consistency, duplicate interactions, dashboard value and information architecture across the Assistant Applications & Claims Management System.

PREPARED	SCOPE	METHOD	CHANGES
30 October 2026	33 pages · 3 roles	Source review + live walkthrough	Read-only · no code edited

Verdict: a broad, working prototype wearing product clothing. The domain coverage is real and the NWU palette is pleasant, but the interface behaves like 33 pages built by different people on different days. There is a design system in `index.css` that almost nothing follows, three status languages, 17 hand-built tables, and a dozen controls that lead to the same place. A user who knows Moodle, Workday, Jira or Stripe will not be lost because the domain is hard — they will be lost because the app breaks the conventions they rely on.

This document is ordered by leverage: what is inconsistent, what is duplicated, what the dashboards are actually worth, and how to restructure the system so that prior experience transfers.

348
raw `gray-*` usages bypassing the theme

17
independent table implementations

13
duplicate interaction pairs with the same result

3
mutually incompatible status colour systems

CONTENTS

1. Executive summary
2. Design consistency
3. Duplicate interactions
4. Dashboard value
5. Information architecture
6. Action plan
7. Evidence index
8. Proposed primitives

1 Executive summary

SCORECARD

DIMENSION	SCORE	JUSTIFICATION
Visual consistency	**3**	Tokens declared, then bypassed 348 times; three page-header styles; three heading scales for one level.
Component reuse	**4**	`Card` is used 178 times, but there are 17 native tables and three badge systems.
Information architecture	**5**	Sensible sections, undermined by hidden destinations, duplicate routes and unreachable pages.
Dashboard value	**4**	Mostly counts without denominators, no trends, actions below the fold or duplicated.
Interaction integrity	**3**	Several primary actions do nothing, or navigate somewhere other than their label promises.
Accessibility	**4**	White-on-yellow at 1.63:1, muted body text at 3.83:1, clipped form values.
Familiarity	**4**	No user menu, breadcrumbs, sortable or paginated tables, toasts, or global search.

Five findings that matter most

1. **The design system is advisory, not enforced.** Every page re-decides typography, spacing, table styling and colour. Consistency is maintained by memory, and memory has failed.
2. **Status is not a shared language.** The same claim is *Pending* in one place and *grey* in another, *Verified* on one screen and *Approved* on the next, and *yellow* on a third. One claim, four truths.
3. **Duplicate entry points hide the real flow.** “Submit Claim”, “Generate New Claim” and “Generate Claim” all lead to a list, not to the wizard; two admin buttons lead to the same screen; two export buttons on one page do the same thing.
4. **Dashboards report, they do not direct.** They answer “what exists” rather than “what needs me now”. Metrics have no period, denominator or delta; the work queue is either absent or below the fold.
5. **One review screen is unreachable.** `ApplicationReview.jsx` (414 lines) is routed at `/application-review/:id` but nothing links to it; the live review lives inline in `ReviewApplications`.

How this was assessed. All 33 pages in `frontend/src/pages` were read, then exercised live against the Docker stack as a student (Naledi), lecturer (Alan) and administrator (Eric). Findings below cite files and line numbers where the cause is unambiguous. Contrast ratios were computed from the theme tokens in `index.css`.

2 Design consistency

2.1 A design system exists on paper only

`frontend/src/index.css` defines `primary`, `neutral`, `success`, `warning`, `error`, `light-grey`, `off-white`, radii and font families. Page code then ignores it:

- **348 uses of raw** `gray-*` — 195 of them `text-gray-*` — against the theme's `neutral`. `CreateBudget.jsx` hardcodes the palette six times: `bg-[#F8F9FA]`, `bg-[#6C3D91]`, `hover:bg-[#5A3280]`.
- **Three page-header bars:** `bg-primary px-8 py-4` on 17 pages, `bg-primary h-16 px-4 md:px-8` on 15, and `bg-primary h-16 px-8` on 2. Only the middle one is responsive; the others keep fixed `px-8` on phones.
- **93 section headings across three sizes** for one semantic level: 18× `text-3xl`, 44× `text-2xl`, 31× `text-xl`. "Overview" is 30px on the lecturer dashboard and 24px on the student dashboard.
- **The title is printed twice.** The page bar says "Work Tracking", then the page repeats "Work Tracking" as an H2. Same pattern on Claims, Applications, Verify Hours and Appointment Approvals.
- **Seventeen tables, one shared component.** Only `AuditLogs` uses `CompactTable`; the other 16 pages build their own, with three header treatments, three row heights and three action styles.
- **Cards look unfinished.** `Card.jsx` applies `border border-dark` — a near-black outline on every surface — which fights the soft purple shadow and makes the UI read as a wireframe.

2.2 Status is the worst offender

There is no single source of truth for what a status means or how it looks:

ONE CLAIM, FOUR PRESENTATIONS

STATUS	STATUSBADGE	CLAIMS VERIFICATION	LECTURER DASHBOARD	EXPORT PAYMENTS
Pending	AMBER	GREY	AMBER ON WHITE	—
Under Review	GREY (MISSING KEY)	RED	AMBER ON WHITE	—
Verified	GREY (MISSING KEY)	GREEN	—	LABELLED "APPROVED"
Approved	GREEN	—	AMBER ON WHITE	GREEN
Rejected	RED	RED	AMBER ON WHITE	—

- `LectureDashboard.jsx:270` hardcodes `bg-warning text-white` for *every* application status. Approved, Rejected and In Review are visually identical, in white on yellow.
- The same claim is *Verified* on Claims Verification and *Approved* on Export Payments. The lifecycle *verified → approved → paid* must be distinct and labelled once.
- "Under Review" rendered as red (`bg-error`) reads as failure rather than progress.

2.3 Numbers and dates are formatted ad hoc

Within a single session the same money appears five ways: `R 45 052,96` (lecturer), `R 563,88` (student dashboard), `R 260.25` (work tracking), `R 107,8` (export) and `R 1 078` (dashboard). Dates appear as *07 Oct 2026*, *07 October 2026* and raw *2026-10-07* in the work-session table. Consolidate into one `formatMoney` and one `formatDate`; ban `toFixed` in page code.

2.4 Feedback, loading and empty states

- **No toast, modal, dialog or drawer primitives exist anywhere.** Feedback is per-page banners; `StudentModules.jsx:169` falls back to `window.confirm`.
- Summary cards render **0 while loading**, so the admin dashboard shows "Pending Claims 0" and then 4. Use skeletons, or `–` as the student dashboard sometimes does.
- Fifteen "Loading…" states are plain text in cards; fifteen empty states are plain text with no call to action.
- Submission does not consistently disable the trigger; the export page instead lets you click and then tells you to select a claim.

2.5 Accessibility is not optional here

COMPUTED CONTRAST AGAINST THE THEME TOKENS

COLOUR PAIR	RATIO	VERDICT	USED FOR
White on success `#28A745`	3.13:1	FAILS AA	Approved / Verified / Paid chips at 14px
White on warning `#FFC107`	1.63:1	UNREADABLE	Every lecturer-dashboard badge
`neutral #78848E` on white	3.83:1	FAILS AA	All secondary and body copy
`neutral` on off-white	3.63:1	FAILS AA	Table meta, helper text
White on error `#DC3545`	4.53:1	BORDERLINE	Rejected chips
White on primary `#6C3D91`	7.74:1	PASSES	Page bars, primary buttons

Beyond colour: status is conveyed by colour alone; the landing hero autoplays video with no `prefers-reduced-motion`; and clipped form values (see §2.6) remove information rather than merely truncating it.

2.6 Brand and vocabulary

- The repository and README say **DACMS**; the navbar, login, landing page and 30 document titles say **AACMS**; `StudentModules.jsx` says DACMS. Pick one.
- **"Demi" and "Assistant"** describe the same person ("Approved Demis", "Total Assistants", "Apply for Assistant"). A student outside NWU will not know "demi".
- **One destination, three names:** sidebar "Assign Duties", page "Assign Assistant Responsibilities", route `/assign-responsibilities`. Same for "Verify Hours" versus "Verify Work Hours".
- **Singular/plural on one screen:** H1 "Appointment Approvals", H2 "Appointment Approval".
- **Truncated values:** in Student Profile, "Person Type" and "Justifiable Reason" both display "Student Assis…" inside 3-column inputs at 802px, so the user cannot read their own selection.

3 Duplicate interactions

These are pairs (or triples) where two different labels already produce the same result — or where the label promises a result the control does not deliver.

CONTROLS THAT LEAD TO THE SAME FLOW RESULT

WHERE	DUPLICATE CONTROLS	DESTINATION / EFFECT	FIX
Student dashboard	"Submit Claim" + "Generate New Claim" card	Both go to `/claims`; neither opens the wizard	One "New claim" → `/claims/new`
Claims page	"Generate New Claim"	`/claims` → second button → `/generate-new-claim`	Link directly; delete one hop
Student dashboard	"Apply to Assistant"	Goes to the applications list, not the apply page	→ positions/apply page
Admin dashboard	"Review Recommendations" + "Approve Appointments"	Both → `/appointment-approvals`	One action, or split real stages
Claims Verification	"Export Payments" (top) + "Export Approved Claims" (bottom)	Both → `/export-payments`	One primary action
Claims Verification	"Claims Summary" + "Verification Summary"	The same four numbers twice on one page	Delete the second block
Export Payments	"Payment Summary" + "Payment Export Summary"	The same selected-claim figures twice	One summary; sticky export bar
Student dashboard	Notifications card + Recent Activity	Both "View all" → `/notifications`	Keep one feed; make activity real audit data
Work Tracking	Appointment card stats + four KPI cards	Identical Worked / Max / Remaining numbers	Keep one
Applications	`ReviewApplications` inline detail + `ApplicationReview` page	Two implementations of the same review screen	Delete the orphan; one pattern
Budgets	`/budget-management` & `/lecturer-budgets`; two detail routes	Same components under role-specific URLs	One role-aware route
App routing	`/notifications` declared three times in `App.jsx`	Same component	Declare once outside role groups
Navigation	Sidebar "Dashboard" + "← Back to Dashboard"	Two ways back to the same page	Keep the sidebar; drop page-level back links

3.1 Orphaned pages and dead code

CODE THAT SHIPS BUT CANNOT BE REACHED OR USED

ARTIFACT	SIZE	STATUS
`pages/ApplicationReview.jsx`	414 lines	Routed at `/application-review/:id`; nothing links to it. The live review lives inline in `ReviewApplications`.
`components/PageNavigation.jsx`	41 lines	Unused component.
`data/claimsData.js`	208 lines	Mock claims, imported by nothing.
`assets/hero.png`, `react.svg`, `vite.svg`, `App.css`	—	Unused.

3.2 Interactions that appear to work but do not

- `VerifyWorkHours.handleApprove()` (lines 180–200) shows "All work hours have been approved" **without calling any API**. Reject is a message telling the user to ask the backend team.
- Admin claim rejection is a "not currently available" message (`ClaimReview.jsx:228`); forgot password throws "not available yet".
- Admin login returns HTTP 202, then `AuthContext.login` calls `response.json()` twice (lines 46–53), so the 2FA flow throws before it can complete.
- `ProtectedRoute` is a no-op, and public registration offers "Administrator" as a self-service role — a security problem wearing a UI costume.

4 Dashboard value

4.1 Student dashboard — 40% signal

Valuable: active positions, hours this month, pending claims, application statuses, monthly hours per module.

Noise: the "Generate Claim" card duplicates quick actions and the Claims page; "Recent Activity" duplicates notifications; a two-column table is used for two modules.

Missing: what needs attention *now* — unverified sessions that block a claim, a claim window closing, outstanding money, profile incompleteness.

Replace with: four KPIs with deltas (hours this month vs last, unverified sessions, claims awaiting action, earnings paid/pending), a "Needs your attention" queue, at most two quick actions, then the claims table. Move monthly hours to the Work page.

4.2 Lecturer dashboard — 55% signal

Valuable: applicants, pending applications, claims summary, sessions awaiting verification, budget utilisation.

Noise: "Approved Demis"; "Hours Allocated 480h" with no cap or denominator; the full budget module list, which belongs on Budgets.

Missing: one ordered approval queue (applications + hours + claims) with counts and age; cap warnings at 80/120 hours per assistant; projected budget overspend.

Replace with: action queue first, portfolio KPIs second, budget health third. Fix the all-yellow badges before anything else.

4.3 Admin dashboard — 30% signal

Valuable: pending claims (4), pending appointments, monthly claims spend.

Noise: "Total Modules / Lecturers / Assistants" (vanity, never changes); **six months of rows including four `R 0` s**; "Monthly Work Sessions" with two months and no CTA; "Administrator Actions" (the sidebar already does this); "Claims Export" card (duplicates a nav destination); and a fifth stat card orphaned in a three-column grid.

Missing: exceptions — budgets near limit, claims pending beyond N days, unverified sessions, payment batch status, spend versus allocated year-to-date, an "as of" timestamp.

Replace with: an attention strip ("4 claims to verify · 0 appointments · 2 budgets near limit"), a KPI row with period selector and deltas, one spend trend chart, top-five budget burn, and recent admin activity from the audit log. Delete the action buttons and export card; make them navigation and header actions.

Bug: `AdminDashboard.jsx:38–55` uses `?? 'null'`, so a missing value literally renders the string *null*.

4.4 Rules for every dashboard

1. Every metric needs a **period** and a **denominator** (spent of allocated, hours of cap).
2. At most **one chart** per dashboard. A table of months with zeros is not a chart.
3. Anything clickable must be the **only** path to that destination, not a third one.

4. Put the **work queue above analytics**: people open dashboards to decide what to do next.

5 Information architecture and familiarity

The current navigation is close, but it hides important destinations and exposes implementation names.

NAVIGATION, BEFORE AND AFTER

ROLE	NOW	PROPOSED
Student	Dashboard, Applications, Modules, Work Tracking, Claims, Profile	Home, Applications, Work, Claims, Modules, Profile
Lecturer	Dashboard, Assistant Listings, Review Applications, Assign Duties, Review Claims, Verify Hours, Budgets	Home, Positions, Applicants, Assistants, Hours, Claims, Budget
Admin	Dashboard, Budget Management, Appointment Approvals, Claims Verification, *Audit hidden as an icon*	Home (queue), Appointments, Claims, Budgets, Payments, Audit — all labelled; Export Payments surfaced

5.1 Conventions users already know

The app omits the furniture that makes Workday, Moodle, Jira and Stripe predictable:

- **User menu** with name, role, settings and logout in the top-right. Today the bar has only a bell; logout sits at the bottom of the side nav; admin audit logs are a mystery icon.
- **Breadcrumbs** on detail pages (*Claims / CLM-2026-01011*), not a "← Back to Dashboard" link on one page only.
- **Data tables** with sort, pagination, bulk select, priority columns and a sticky action column. Today the Claims table hides Amount, Status and "View Details" off-screen; Export wraps Claim ID and Date onto three lines.
- **One detail pattern** — drawer or route — with a status stepper. `ApplicationDetail` already has "Approval Progress"; generalise it.
- **Global search or command palette** (Ctrl+K) for students, modules, claims and references.
- **Toasts and confirm dialogs** instead of banners and `window.confirm`.
- **Settings in the user menu**, not a "Comfortable view" button in the Audit Logs page header.
- **Empty states with a CTA** ("No applications yet → Apply for an assistant position").

6 Action plan

The plan runs as a one-week hardening sprint followed by two two-week sprints. Each sprint ends with a demo against its exit criteria; nothing is "done" because the code merged.

SPRINT SEQUENCE

SPRINT	WHEN	GOAL	SCOPE	EXIT CRITERIA
Sprint 0	Week 1	Tell the truth	Six deletion-first steps: auth guard, one status language, honest CTAs, duplicate summaries, dead controls, dead code.	No primary action is a no-op; one status language; no unreachable route; every P0 check passes.
Sprint 1	Weeks 2–3	One system	`ui/` primitives, `gray-*` codemod, one page header and heading scale, `DataTable` across all 17 tables, card restyle, formatters, one detail pattern and the merged claims workspace.	Zero raw `gray-*` in pages; one header component; every data table is a `DataTable`; no duplicated detail screens.
Sprint 2	Weeks 4–5	One product	Three dashboards rebuilt around the work queue, command palette / breadcrumbs / user menu, profile single-save, responsive pass.	Each dashboard answers "what do I do now"; 390 / 768 / 1280 pass; settings live in the user menu.

Sequencing. Sprint 0 ships in week one even if everything else slips: it removes the lies and the dead ends that make every other improvement look cosmetic. Sprint 1 must not start before the status model in Sprint 0 is settled, because `StatusBadge` and every Sprint 1 primitive consume it. The two-week sprints assume one frontend developer plus part-time design review; treat the ranges as indicative, double them for a solo effort, and split Sprint 1 across two developers only along the table/header seam.

P0 Sprint 0 — Correctness and trust

Sprint 0 is a demolition sprint, not a design sprint. One rule governs every item: **keep the implementation that is already wired to the API and delete the other one**. Do not create a `ui/` folder, do not rename components, do not touch the 348 `gray-*` classes, do not restyle `Card`, and do not rebuild a dashboard. The sprint is roughly 22 developer-hours and should read as a small diff of deletions and one-line fixes.

Step 1 · Make the guard real 4 H

Files: `routes/ProtectedRoute.jsx`, `contexts/AuthContext.jsx`, `pages/Login.jsx`.

- Replace the demo body of `ProtectedRoute` — currently `return <Outlet />` — with the check below. Eight lines, no helper module, no new dependency.

```
const HOME = {
  student: '/student-dashboard',
  lecturer: '/lecturer-dashboard',
  admin: '/admin-dashboard',
};

const ProtectedRoute = ({ requiredRoles = [] }) => {
  const { user, loading } = useAuth();
  if (loading) return null;
  if (!user) return <Navigate to="/login" replace />;
  if (requiredRoles.length && !requiredRoles.includes(user.role)) {
    return <Navigate to={HOME[user.role] ?? '/login'} replace />;
  }
  return <Outlet />;
};
```

- `AuthContext.login` reads the response body twice: the 202 branch calls `response.json()` at line 47 and line 53 calls it again, which throws. Read once, branch, return.

```
const data = await response.json();

if (response.status === 202) {
  navigate('/2fa', {
    replace: true,
    state: { from: data.requiresSetup ? 'setup' : 'login', user: data.user },
  });
  return null;
}
```

- Delete `{ value: 3, label: 'Administrator' }` from `ROLE_OPTIONS` in `Login.jsx` (line 8). Public registration must not mint admins.
- **Known gap, do not hide it:** `POST /users/create` still accepts `role_id: 3`, and `backend/tests/api-route-smoke-test.js` creates its admin through that public endpoint. The frontend change stops honest users, not attackers. Close it by seeding the test admin directly and rejecting role 3 on the public route; until then, registration is not actually locked down.

Done when: logged out, `/admin-dashboard` redirects to `/login`; logged in as a student, it redirects to `/student-dashboard`; admin login reaches `/2fa` instead of throwing.

Step 2 · One status language `6 H`

Files: `index.css`, `components/StatusBadge.jsx`, `pages/ClaimsVerification.jsx`, `pages/LectureDashboard.jsx`, `pages/ExportPayments.jsx`, `backend/src/repositories/AdminRepository.js`.

- Add four tint pairs to `@theme` in `index.css`. These are the only new tokens Sprint 0 may introduce; every existing status colour is replaced by them.

```
--color-status-amber: #8A5A00;      --color-status-amber-soft: #FFF4D6;
--color-status-progress: #3B4E9B;   --color-status-progress-soft: #E8ECF9;
--color-status-done: #176B3A;       --color-status-done-soft: #E7F6EC;
--color-status-danger: #B42318;     --color-status-danger-soft: #FDECEA;
```

- Rewrite `StatusBadge.jsx` in place. Keep the filename and default export so the 18 existing imports keep working. One map, one fallback, no inline colour logic anywhere else.

```
const TONES = {
  pending: 'bg-status-amber-soft text-status-amber',
  review: 'bg-status-progress-soft text-status-progress',
  done: 'bg-status-done-soft text-status-done',
  danger: 'bg-status-danger-soft text-status-danger',
};

const STATUS = {
  Pending: 'pending',
  'In Review': 'review',
  'Under Review': 'review',
  'Approved by Lecturer': 'done',
  Verified: 'done',
  Approved: 'done',
  Paid: 'done',
  Rejected: 'danger',
  'Rejected by Lecturer': 'danger',
};

const StatusBadge = ({ status }) => {
  const label = String(status ?? '').trim() || 'Unknown';
  const tone = TONES[STATUS[label]] ?? 'bg-light-grey text-dark';
  return (
    <span className={`rounded-full px-3 py-1 text-sm font-semibold ${tone}`}>
      {label}
    </span>
  );
};
```

- Delete `claimStatusClass` in `ClaimsVerification.jsx` (lines 73–88) and the inline span at 391–395; use `<StatusBadge status={claim.claim_status} />` and import it.
- Replace the hardcoded yellow span in `LectureDashboard.jsx` (line 270) with `<StatusBadge status={app.ApplicationStatus} />`; import it.
- `AdminRepository.getClaimsForExport` aliases `'Approved' AS status` (line 455) while selecting only `ClaimStatus = 'Verified'`. Return the real status: change the alias to `c."ClaimStatus" AS status`.
- In `ExportPayments.jsx`, delete the status filter entirely: the state at line 11, the select at 250–265 and the `matchesStatus` clause at line 52. The endpoint only returns verified claims, so the filter can only hide everything.

Done when: the same claim shows the same word and the same tone on Claims Verification, Lecturer Dashboard, Review Claims and Export Payments; `grep -r "claimStatusClass\|bg-warning text-white" src` returns nothing.

Step 3 · CTAs land where they say `2 H`

Files: `pages/Dashboard.jsx`, `pages/AdminDashboard.jsx`.

- `Dashboard.jsx:130` "Apply to Assistant" → `/apply-for-assistant`.
- `Dashboard.jsx:137` "Submit Claim" → `/generate-new-claim`.
- `Dashboard.jsx:419` "Generate New Claim" → `/generate-new-claim` (the card itself is deleted in Step 4).
- `AdminDashboard.jsx:328–333` delete "Review Recommendations"; "Approve Appointments" goes to the same page.

Done when: no button labelled Apply, Submit or Generate opens a list that requires a second click to reach the form.

Step 4 · Delete duplicated summaries `5 H`

- `ClaimsVerification.jsx:417–476`: delete the whole "Verification Summary" section. It repeats the four stat cards at 214–277 and contains the second Export button.
- `ExportPayments.jsx:299–350`: delete "Payment Summary"; keep "Payment Export Summary" (515+) as the single pre-export confirmation.

- `WorkTracking.jsx:554–605` : delete the four-card stats grid. Worked, Maximum and Remaining already appear in the appointment card at 347–366; move Estimated Earnings into that card as a fourth column instead of a second summary.
- `AdminDashboard.jsx` : delete the "Administrator Actions" section (315–348) and the "Claims Export" card (351 onward). The sidebar already owns navigation; the dashboard is not a second navigation.
- `Dashboard.jsx:406–470` : delete the "Generate Claim" and "Recent Activity" section. The quick action covers claim creation and the notifications card already covers activity.

Done when: each number and each action appears once per page.

Step 5 · Remove controls that cannot work `3 H`

- `VerifyWorkHours.jsx` : delete `handleApprove` (180–200), `handleReject` (202–213), the commented-out decision card (388 onward) and the state only they use (`rejectionReason` , `showRejectBox`). The per-row Verify/Unverify actions at 357–379 already call the API and are the only hours-approval UI.
- `ClaimReview.jsx` : wrap the "Reject Claim" button (693–700) in `{isLecturerReview && ...}` and delete the admin branch of `handleReject` (227–229). Admin can verify a claim; it cannot reject one, so it must not offer the button or an "ask the backend team" message.
- `StudentModules.jsx:169` uses `window.confirm` . It is native, it works, and replacing it is polish. Leave it.

Done when: no visible control ends in a message that says the action is unavailable.

Step 6 · Delete dead code, declare one route `2 H`

- Delete `pages/ApplicationReview.jsx` (414 lines), its import (`App.jsx:26`) and its route (line 88). The live review is inline in `ReviewApplications.jsx` .
- Delete `components/PageNavigation.jsx` , `data/claimsData.js` , `assets/react.svg` , `assets/vite.svg` , `assets/hero.png` and `src/App.css` ; nothing imports them.
- Declare `/notifications` once in a shared group instead of three times (lines 75, 102, 130):

```
<Route element={<ProtectedRoute requiredRoles={['student', 'lecturer', 'admin']} />}>
  <Route path="/notifications" element={<Notifications />} />
</Route>
```

Keep `/review-claim/:id` . `ReviewClaims.jsx:93` navigates to it and `ClaimReview` serves both roles; it is not an orphan.

Done when: nothing in `src` imports a deleted file and `npm run build` passes.

One implementation per deliverable

WHAT SURVIVES SPRINT 0

DELIVERABLE	TODAY	KEEP	DELETE
Status rendering	`StatusBadge`, `claimStatusClass`, hardcoded yellow span	`StatusBadge.jsx`	Both inline maps
Application review	Inline detail in `ReviewApplications` plus `ApplicationReview.jsx`	Inline detail	414-line page, route, import
Claim review	`ClaimReview` shared by lecturer and admin	`ClaimReview` and `/review-claim/:id`	Nothing
Hours approval	Per-row Verify/Unverify plus dead bulk handlers	Per-row actions	Bulk handlers and commented card
Payment export	Header link, bottom button, two summaries	Header link and export summary	Bottom button, Payment Summary
New claim	Quick action plus “Generate Claim” card	Quick action	Generate Claim card
Appointments	Two admin buttons, one destination	Approve Appointments	Review Recommendations
Notifications route	Declared three times	One shared route	Three declarations

Out of scope on purpose. No `ui/` primitives, no `DataTable`, no `AppLayout`, no token codemod, no card restyle, no dashboard rebuild, no page merges, no new dependencies. Those are Sprint 1 and 2. If a Sprint 0 change starts to look like a refactor, stop and leave it for the next sprint.

Acceptance checklist

- Logged out, every role route redirects to `/login`; cross-role access lands on the user’s own dashboard.
- One status word and one tone per state across all four screens; no white on warning or success.
- Apply, Submit Claim and Export each reach their action in one click.
- Approve and Reject exist only where the API supports them; no “ask the backend team” message remains.
- `ApplicationReview`, `PageNavigation`, `claimsData`, the unused assets and the commented decision card are gone, and every remaining route is reachable from the UI.
- `npm run build` passes and the diff contains no new components.

P1 Sprint 1 — Adopt the design system

1. Build `ui/` primitives: `AppLayout`, `PageHeader`, `Button`, `DataTable`, `Section`, `FormField`, `Toast`, `ConfirmDialog`, `Money` / `Date` formatters, `EmptyState`. `StatusBadge` already exists from Sprint 0; do not recreate it.
2. **Codemod `gray-*` to semantic tokens**; remove hardcoded hex. One page-header component across all 33 pages; one heading scale (24 page, 18 section, 16 card, 14 body, 12 caption).
3. Replace all 17 tables with `DataTable`: nowrap IDs and currency, sortable columns, pagination, mobile card fallback.
4. Restyle `Card` (no black border; 1px `#E5E7EB`; 16/24 padding) and stop wrapping tables in padded cards.
5. Standardise currency and dates; remove `toFixed` from pages.
6. Replace duplicated list/detail implementations with one routed detail pattern; merge Create/Edit Budget into one form.
7. Merge Claims Verification and Export Payments into a “Claims” workspace with *Verify* / *Export* tabs, or keep them separate without duplicated summaries and cross-links.

P2 Sprint 2 — Product maturity

1. Rebuild the three dashboards around **work queue → KPIs with deltas → one trend → recent activity**, with a period selector.

2. Add command palette, breadcrumbs, user menu, saved filters, bulk approve, skeletons.
3. Profile: one form with section tabs and **one** save (today it is six independently saved sections), full-width fields, no truncation.
4. Responsive pass at 390 / 768 / 1280: fixed `px-8` headers, three-column forms at 768, clipped tables, and the landing page whose two 180px buttons plus 40px inline margins exceed a 375px viewport.

A Evidence index

ISSUES BY FILE

FILE	LINE	ISSUE
`index.css`	5–23	Theme tokens declared; bypassed throughout page code.
`components/Card.jsx`	5	`border border-dark` produces a near-black outline on every card.
`components/StatusBadge.jsx`	5–12	Missing `Verified`, `Under Review`, `Pending Review`; white on success fails AA.
`pages/ClaimsVerification.jsx`	73–88	Local status map: Pending grey, Under Review red — conflicts with `StatusBadge`.
`pages/LectureDashboard.jsx`	270	All application statuses hardcoded `bg-warning text-white`.
`pages/AdminDashboard.jsx`	38–55	`?? 'null'` renders the literal string “null”.
`pages/AdminDashboard.jsx`	328–340	Two buttons, one destination (`/appointment-approvals`).
`pages/Dashboard.jsx`	122–142, 418–423	Claim CTAs land on `/claims`, not the wizard; “Apply to Assistant” lands on the list.
`pages/Claims.jsx`	65–67, 196	`toFixed` currency; `min-w-[900px]` table clips Amount, Status and action.
`pages/VerifyWorkHours.jsx`	180–213	Approve only flashes a message; Reject is unimplemented.
`pages/ClaimReview.jsx`	228	Admin rejection is an availability message.
`contexts/AuthContext.jsx`	46–53	HTTP 202 branch reads `response.json()` twice.
`routes/ProtectedRoute.jsx`	3–6	Authentication is a no-op in demo mode.
`pages/Login.jsx`	5–9, 274–284	Public registration lets anyone choose “Administrator”.
`pages/StudentProfile.jsx`	354	Six sections with independent saves; 3-column inputs truncate values.
`pages/CreateBudget.jsx`	172–443	Hardcoded hex colours duplicate theme tokens.
`pages/StudentModules.jsx`	169	`window.confirm` used for a destructive action.
`App.jsx`	75, 102, 130	`/notifications` declared three times.
`App.jsx`	88	`/application-review/:id` has no inbound link.

B Proposed component contracts

The smallest set that removes most of the inconsistency, with the API shape the page code should consume:

```
// One shell, one header, one title
<AppLayout role="admin" title="Claims" actions={<Button>New claim</Button>}>
  <PageHeader breadcrumbs={[...]} />
  <Section title="Awaiting verification">
    <DataTable
      columns={columns} rows={claims}
      sortable paginated selectable
      empty={<EmptyState action="Create the first claim" />}
      loading={loading} />
  </Section>
</AppLayout>

// One status vocabulary, one tone map
<StatusBadge status="Verified" />   // label + tone from STATUS

// One money/date contract
formatMoney(563.88)  // "R 563,88"
formatDate(iso)      // "07 Oct 2026"
```

Migration order: tokens and typography → `Button` and `Card` → `StatusBadge` → `DataTable` → `AppLayout` and `PageHeader` → forms and feedback. Each step is independently shippable, and each reduces the diff of the next.

Bottom line. The app has the right pages and the right data; it lacks a spine. Adopt a shell, a status model, a table and a feedback system; delete the duplicate entry points and dead review pages; rebuild the dashboards around “what do I need to do now” instead of “what can we count”. That alone moves it from “student project that works” to “system an NWU administrator can trust”.

Prepared 30 October 2026 · Read-only audit · No application code was modified.